Snowflake w praktyce

Bezpieczeństwo danych klientek sieci salonów: RBAC, użytkownicy, Dynamic Data Masking, Row Access Policies i Time Travel

Warsztat praktyczny · 4 moduły z weryfikacją

Poziom: średniozaawansowany (SQL, podstawy Snowflake, Time Travel)

Spis treści

1. Scenariusz i plan warsztatu ... 3
1.1. Scenariusz biznesowy ... 3
1.2. Macierz dostępu – cel warsztatu ... 3
1.3. Architektura ról ... 3
1.4. Agenda ... 4
1.5. Wymagania ... 4
1.6. Oczekiwane liczby ... 4
2. Przygotowanie środowiska (start od zera) ... 6
3. Moduł 1. RBAC – role, uprawnienia i użytkownicy ... 9
Role systemowe – kto co robi ... 9
Weryfikacja modułu 1 ... 12
4. Moduł 2. Dynamic Data Masking – maskowanie kolumn ... 14
Weryfikacja modułu 2 ... 16
5. Moduł 3. Row Access Policies – bezpieczeństwo wierszy ... 17
Weryfikacja modułu 3 ... 18
6. Moduł 4. Time Travel – śledztwo i odtwarzanie po incydencie ... 20
Weryfikacja modułu 4 ... 22
7. Podsumowanie i sprzątanie ... 23
7.1. Co zbudowaliśmy ... 23
7.3. Pytania kontrolne ... 23
7.4. Sprzątanie ... 23

1. Scenariusz i plan warsztatu

1.1. Scenariusz biznesowy

GlowUp to sieć czterech salonów kosmetycznych: Warszawa (WAW), Kraków (KRK), Gdańsk (GDA) i Poznań (POZ). Wszystkie salony korzystają ze wspólnej bazy klientek w Snowflake. Baza zawiera dane osobowe (e-mail, telefon, PESEL, data urodzenia), zgodę marketingową oraz notatki o alergiach i przeciwwskazaniach. Te ostatnie to dane o zdrowiu, czyli szczególna kategoria danych w rozumieniu art. 9 RODO.

Do tej pory wszyscy pracownicy logowali się jedną wspólną rolą i widzieli wszystko. Inspektor ochrony danych (IOD) zlecił wdrożenie zasady **minimalnego dostępu**:

- każdy pracownik ma imienne konto i role zgodne ze stanowiskiem, a konto techniczne narzędzia BI jest oddzielone od kont ludzi,
- kierowniczka salonu widzi tylko klientki swojego salonu, a dyrektor regionu – salonów ze swojego regionu,
- centrala analizuje wszystkie dane, ale bez danych identyfikujących i bez danych o zdrowiu,
- marketing widzi adres e-mail wyłącznie klientek, które wyraziły zgodę marketingową,
- każdą pomyłkę pracownika (np. UPDATE bez WHERE) da się zbadać i odwrócić, a samo odtwarzanie danych nie omija zabezpieczeń.

Każdy moduł kończy się zapytaniem weryfikacyjnym, które zwraca listę testów ze statusem **PASS** lub **FAIL**.

1.2. Macierz dostępu – cel warsztatu

Tak ma wyglądać dostęp do tabeli `CRM.CLIENTS` po zakończeniu modułu 3:

Rola	Wiersze	E-mail	Telefon	PESEL	Data ur.	Alergie
DPO (IOD)	Wszystkie	Pełny	Pełny	Pełny	Pełna	Tak
MGR_<salon>	Swój salon	Pełny	Pełny	`*******1234`	Pełna	Tak
REGION_NORTH	GDA + POZ	Pełny	Pełny	`*******1234`	Pełna	Tak
ANALYST	Wszystkie	`*****@wp.pl`	`*** *** 123`	`**********`	1 stycznia	NULL
MARKETING	Wszystkie	Tylko ze zgodą	`*** *** 123`	`**********`	1 stycznia	NULL
SYSADMIN (właściciel)	Brak	Ukryty	`*** *** 123`	`**********`	1 stycznia	NULL

Nazwy ról w tabeli podano bez prefiksu `GLOWUP_`.

Ostatni wiersz jest celowy: **właściciel tabeli nie omija polityk**. Administrator infrastruktury nie musi mieć wglądu w dane osobowe.

1.3. Architektura ról

Typ roli	Role	Przeznaczenie
Systemowe	`USERADMIN`, `SECURITYADMIN`, `SYSADMIN`, `ACCOUNTADMIN`	Tworzenie użytkowników i ról, nadawanie uprawnień, własność obiektów, uprawnienia na poziomie konta
Dostępowe (access)	`GLOWUP_READ`, `GLOWUP_WRITE`	Paczki uprawnień do obiektów (SELECT, DML). Nigdy nie nadawane bezpośrednio ludziom
Funkcjonalne	`GLOWUP_ANALYST`, `GLOWUP_MARKETING`, `GLOWUP_DPO`, `GLOWUP_SALON_MANAGER`	Stanowiska; dziedziczą role dostępowe

Typ roli	Role	Przeznaczenie
Zakresowe	`GLOWUP_MGR_WAW`, `_KRK`, `_GDA`, `_POZ`, `GLOWUP_REGION_NORTH`	Wyznaczają, które salony widzi użytkownik (polityka wierszy)
Governance	`GLOWUP_GOVERNANCE`	Tworzy i podpina polityki maskowania i wierszy (rozdzielenie obowiązków)

Hierarchia: `GLOWUP_READ → GLOWUP_WRITE → SYSADMIN`, `GLOWUP_WRITE → GLOWUP_SALON_MANAGER → GLOWUP_MGR_<salon>`, `GLOWUP_MGR_GDA + GLOWUP_MGR_POZ → GLOWUP_REGION_NORTH` (strzałka oznacza „jest nadana roli").

1.4. Agenda

Część	Weryfikacja
Wprowadzenie i przygotowanie środowiska (dane klientek i wizyt)	Test startowy
Moduł 1. RBAC – role, uprawnienia i użytkownicy	5 testów
Moduł 2. Dynamic Data Masking – maskowanie kolumn	5 testów
Moduł 3. Row Access Policies – bezpieczeństwo wierszy	5 testów
Moduł 4. Time Travel – śledztwo i odtwarzanie po incydencie	5 testów
Podsumowanie i sprzątanie	–

1.5. Wymagania

- Konto Snowflake w edycji **Enterprise** lub wyższej (konto trial domyślnie ją ma). Polityki maskowania, polityki wierszy i retencja Time Travel dłuższa niż 1 dzień nie działają w edycji Standard.
- Użytkownik z rolą **ACCOUNTADMIN** (na koncie trial – użytkownik, który je założył).
- Pracujemy w **jednym arkuszu**. Testy przełączają role poleceniem `USE ROLE`, a wyniki zapisują w zmiennych sesji (`SET`), które przetrwają zmianę roli.
- Dane są w pełni fikcyjne. Numery PESEL mają poprawny format, ale nie mają poprawnej sumy kontrolnej.

Najważniejsza pułapka: role drugorzędne

Nowi użytkownicy Snowflake mają domyślnie włączone wszystkie role drugorzędne (`DEFAULT_SECONDARY_ROLES = ('ALL')`). Wtedy po `USE ROLE GLOWUP_ANALYST` w sesji nadal aktywne są wszystkie inne role użytkownika – także `GLOWUP_DPO`. Funkcja `IS_ROLE_IN_SESSION()` w politykach zwraca wtedy TRUE dla każdej z nich i testy pokazują pełne dane.

Dlatego na początku każdego modułu i każdej weryfikacji wykonujemy `USE SECONDARY ROLES NONE;`.

1.6. Oczekiwane liczby

Reguły generatora są stałe: salon = `MOD(rn,4)`, zgoda marketingowa co 3. klientka, notatka o alergii co 10. klientka, każda klientka ma dokładnie 5 wizyt.

Obiekt / rola	Oczekiwany wynik
CRM.CLIENTS	1200 klientek, po 300 w salonie, 400 zgód marketingowych, 120 notatek o alergiach
SALES.VISITS	6000 wizyt, po 1500 w salonie

Obiekt / rola	Oczekiwany wynik
Klientka testowa C00030	Salon GDA, ma zgodę marketingową, ma notatkę o alergii
Klientka testowa C00010	Salon GDA, bez zgody marketingowej, ma notatkę o alergii
Użytkownicy GlowUp	8 kont (7 osób + 1 konto techniczne), 1 zablokowane
MGR_KRK / REGION_NORTH / SYSADMIN	300 / 600 / 0 klientek
Zgody w salonie KRK	100 (incydent w module 4 je kasuje, odtwarzanie przywraca)

2. Przygotowanie środowiska (start od zera)

Skrypt tworzy warehouse, bazę z trzema schematami, cennik usług, 1200 klientek i 6000 wizyt. Dane robocze generatora zawierają dane osobowe bez żadnej ochrony, dlatego na końcu usuwamy schemat UTIL – dobry nawyk przy pracy z danymi osobowymi.

Skopiuj skrypt do nowego arkusza i uruchom go w całości (**Run All**).

Krok 0.1. Warehouse, baza i schematy

```
USE ROLE SYSADMIN;

CREATE OR REPLACE WAREHOUSE GLOWUP_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE DATABASE GLOWUP;
CREATE SCHEMA GLOWUP.CRM;     -- klientki
CREATE SCHEMA GLOWUP.SALES;   -- wizyty i cennik
CREATE SCHEMA GLOWUP.UTIL;    -- tymczasowe dane generatora (usuwane na końcu)

USE WAREHOUSE GLOWUP_WH;
USE DATABASE GLOWUP;
```

Krok 0.2. Salony i cennik usług

```
CREATE OR REPLACE TABLE GLOWUP.SALES.SALONS (
  salon_id VARCHAR(3), salon_name VARCHAR(60), city VARCHAR(30), region VARCHAR(10)
);
INSERT INTO GLOWUP.SALES.SALONS VALUES
  ('WAW', 'GlowUp Warszawa Mokotow', 'Warszawa', 'CENTRUM'),
  ('KRK', 'GlowUp Krakow Kazimierz',  'Krakow',   'POLUDNIE'),
  ('GDA', 'GlowUp Gdansk Oliwa',      'Gdansk',   'POLNOC'),
  ('POZ', 'GlowUp Poznan Jezyce',     'Poznan',   'POLNOC');

CREATE OR REPLACE TABLE GLOWUP.SALES.SERVICES (
  service_id INTEGER, service_name VARCHAR(60), price NUMBER(8,2)
);
INSERT INTO GLOWUP.SALES.SERVICES VALUES
  (1, 'Manicure hybrydowy',        140.00),
  (2, 'Pedicure leczniczy',        160.00),
  (3, 'Oczyszczanie wodorowe',     220.00),
  (4, 'Mezoterapia mikroiglowa',   450.00),
  (5, 'Henna brwi i rzes',          80.00),
  (6, 'Masaz twarzy Kobido',       260.00),
  (7, 'Depilacja laserowa',        390.00),
  (8, 'Peeling kawitacyjny',       150.00);
```

Krok 0.3. Dane bazowe klientek

Wartości losowe zapisujemy najpierw do tabeli roboczej. Dzięki temu imię i nazwisko użyte w adresie e-mail są tymi samymi wartościami, które trafią do kolumn.

```
CREATE OR REPLACE TABLE GLOWUP.UTIL.CLIENTS_BASE AS
WITH g AS (
  SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) AS rn
  FROM TABLE(GENERATOR(ROWCOUNT => 1200))
)
SELECT
  rn,
  GET(ARRAY_CONSTRUCT('Anna', 'Katarzyna', 'Magdalena', 'Agnieszka', 'Joanna',
                      'Monika', 'Natalia', 'Aleksandra', 'Paulina', 'Karolina'),
      UNIFORM(0, 9, RANDOM()))::VARCHAR                              AS first_name,
  GET(ARRAY_CONSTRUCT('Nowak', 'Kowalska', 'Wisniewska', 'Wojcik', 'Kaminska',
```

```
                      'Lewandowska', 'Zielinska', 'Szymanska', 'Wozniak', 'Dabrowska'),
      UNIFORM(0, 9, RANDOM()))::VARCHAR                              AS last_name,
  GET(ARRAY_CONSTRUCT('gmail.com', 'wp.pl', 'onet.pl', 'o2.pl', 'interia.pl'),
      UNIFORM(0, 4, RANDOM()))::VARCHAR                              AS email_domain,
  DATEADD(day, UNIFORM(0, 15000, RANDOM()), '1960-01-01'::DATE)      AS birth_date,
  UNIFORM(500, 899, RANDOM())                                        AS ph1,
  UNIFORM(0, 999, RANDOM())                                          AS ph2,
  UNIFORM(0, 999, RANDOM())                                          AS ph3,
  UNIFORM(0, 99999, RANDOM())                                        AS pesel_tail,
  GET(ARRAY_CONSTRUCT('Uczulenie na lateks', 'Alergia na nikiel',
                      'Atopowe zapalenie skory', 'Nadwrazliwosc na kwas hialuronowy'),
      UNIFORM(0, 3, RANDOM()))::VARCHAR                              AS allergy_text,
  DATEADD(day, -UNIFORM(30, 1500, RANDOM()), '2026-11-01'::TIMESTAMP_NTZ) AS created_at
FROM g;
```

Krok 0.4. Tabela klientek

```
CREATE OR REPLACE TABLE GLOWUP.CRM.CLIENTS (
  client_id         VARCHAR(10),
  first_name        VARCHAR(40),
  last_name         VARCHAR(40),
  email             VARCHAR(120),
  phone             VARCHAR(20),
  pesel             VARCHAR(11),
  birth_date        DATE,
  home_salon_id     VARCHAR(3),
  marketing_consent BOOLEAN,
  allergy_notes     VARCHAR(200),
  created_at        TIMESTAMP_NTZ
);

INSERT INTO GLOWUP.CRM.CLIENTS
SELECT 'C' || LPAD(rn::VARCHAR, 5, '0'),
       first_name,
       last_name,
       LOWER(first_name || '.' || last_name || rn || '@' || email_domain),
       '+48 ' || ph1 || ' ' || LPAD(ph2::VARCHAR, 3, '0')
              || ' ' || LPAD(ph3::VARCHAR, 3, '0'),
       TO_CHAR(birth_date, 'YYMMDD') || LPAD(pesel_tail::VARCHAR, 5, '0'),
       birth_date,
       DECODE(MOD(rn, 4), 0, 'WAW', 1, 'KRK', 2, 'GDA', 'POZ'),
       MOD(rn, 3) = 0,
       IFF(MOD(rn, 10) = 0, allergy_text, NULL),
       created_at
FROM GLOWUP.UTIL.CLIENTS_BASE;
```

Krok 0.5. Wizyty – 5 na klientkę

```
CREATE OR REPLACE TABLE GLOWUP.UTIL.VISITS_BASE AS
WITH g AS (
  SELECT ROW_NUMBER() OVER (ORDER BY SEQ4()) AS rn
  FROM TABLE(GENERATOR(ROWCOUNT => 6000))
)
SELECT rn,
       MOD(rn - 1, 1200) + 1        AS client_rn,
       UNIFORM(1, 8, RANDOM())      AS service_id,
       UNIFORM(0, 364, RANDOM())    AS days_ago,
       UNIFORM(1, 3, RANDOM())      AS pay_no
FROM g;

CREATE OR REPLACE TABLE GLOWUP.SALES.VISITS AS
SELECT 'V' || LPAD(b.rn::VARCHAR, 6, '0')                  AS visit_id,
       c.client_id,
       c.home_salon_id                                     AS salon_id,
       s.service_name,
       s.price                                             AS amount,
       DATEADD(day, -b.days_ago, '2026-11-01'::DATE)       AS visit_date,
       DECODE(b.pay_no, 1, 'KARTA', 2, 'BLIK', 'GOTOWKA')  AS payment_method
```

```
FROM GLOWUP.UTIL.VISITS_BASE b
JOIN GLOWUP.CRM.CLIENTS c
  ON c.client_id = 'C' || LPAD(b.client_rn::VARCHAR, 5, '0')
JOIN GLOWUP.SALES.SERVICES s ON s.service_id = b.service_id;

-- Dane robocze zawierają dane osobowe bez ochrony - usuwamy je
DROP SCHEMA GLOWUP.UTIL;
```

Krok 0.6. Test startowy

```
SELECT 'CLIENTS: 1200 klientek, po 300 w kazdym z 4 salonow' AS test,
       IFF((SELECT COUNT(*) || '/' || COUNT(DISTINCT home_salon_id) || '/' ||
                   MIN(cnt) || '/' || MAX(cnt)
            FROM (SELECT home_salon_id, COUNT(*) OVER (PARTITION BY home_salon_id) AS cnt
                  FROM GLOWUP.CRM.CLIENTS)) = '1200/4/300/300', 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '400 zgod marketingowych i 120 notatek o alergiach',
       IFF((SELECT COUNT_IF(marketing_consent) || '/' || COUNT(allergy_notes)
            FROM GLOWUP.CRM.CLIENTS) = '400/120', 'PASS', 'FAIL')
UNION ALL
SELECT 'VISITS: 6000 wizyt, kazda klientka ma 5',
       IFF((SELECT COUNT(*) || '/' || COUNT(DISTINCT client_id)
            FROM GLOWUP.SALES.VISITS) = '6000/1200', 'PASS', 'FAIL')
UNION ALL
SELECT 'Schemat UTIL z danymi roboczymi zostal usuniety',
       IFF((SELECT COUNT(*) FROM GLOWUP.INFORMATION_SCHEMA.SCHEMATA
            WHERE schema_name = 'UTIL') = 0, 'PASS', 'FAIL');
```

Oczekiwany wynik: cztery wiersze ze statusem PASS.

3. Moduł 1. RBAC – role, uprawnienia i użytkownicy

Cel modułu

Rozumiesz model RBAC w Snowflake (uprawnienia nadaje się rolom, role użytkownikom), wiesz, która rola systemowa do czego służy, budujesz hierarchię ról dostępowych i funkcjonalnych, używasz uprawnień FUTURE, tworzysz konta ludzi i konto techniczne, blokujesz konto po odejściu pracownika i potrafisz audytować nadania.

Role systemowe – kto co robi

Rola	Odpowiedzialność
ACCOUNTADMIN	Najwyższa rola konta (parametry konta, rozliczenia). Używana tylko wyjątkowo, nigdy jako rola domyślna
SECURITYADMIN	Ma uprawnienie MANAGE GRANTS – nadaje i odbiera uprawnienia do dowolnych obiektów; dziedziczy USERADMIN
USERADMIN	Tworzy użytkowników i role (CREATE USER, CREATE ROLE)
SYSADMIN	Tworzy i posiada bazy, schematy, tabele, warehouse. Zalecany „dach” dla ról niestandardowych
PUBLIC	Rola automatycznie nadana każdemu użytkownikowi

Różnica względem PostgreSQL/Oracle: w Snowflake uprawnienia do obiektów nadaje się wyłącznie rolom – nie da się nadać SELECT bezpośrednio użytkownikowi. Każdy obiekt ma dokładnie jedną rolę-właściciela (OWNERSHIP), a nie użytkownika.

Krok 1.1. Tworzenie ról (USERADMIN)

```
USE SECONDARY ROLES NONE;
USE ROLE USERADMIN;

-- Role dostępowe
CREATE OR REPLACE ROLE GLOWUP_READ  COMMENT = 'Access: odczyt CRM i SALES';
CREATE OR REPLACE ROLE GLOWUP_WRITE COMMENT = 'Access: odczyt i DML w CRM i SALES';

-- Role funkcjonalne
CREATE OR REPLACE ROLE GLOWUP_ANALYST       COMMENT = 'Analityk centrali';
CREATE OR REPLACE ROLE GLOWUP_MARKETING     COMMENT = 'Dzial marketingu';
CREATE OR REPLACE ROLE GLOWUP_DPO           COMMENT = 'Inspektor ochrony danych (IOD)';
CREATE OR REPLACE ROLE GLOWUP_SALON_MANAGER COMMENT = 'Kierownik salonu - wspolne';

-- Role zakresowe (który salon)
CREATE OR REPLACE ROLE GLOWUP_MGR_WAW      COMMENT = 'Kierownik salonu Warszawa';
CREATE OR REPLACE ROLE GLOWUP_MGR_KRK      COMMENT = 'Kierownik salonu Krakow';
CREATE OR REPLACE ROLE GLOWUP_MGR_GDA      COMMENT = 'Kierownik salonu Gdansk';
CREATE OR REPLACE ROLE GLOWUP_MGR_POZ      COMMENT = 'Kierownik salonu Poznan';
CREATE OR REPLACE ROLE GLOWUP_REGION_NORTH COMMENT = 'Dyrektor regionu Polnoc (GDA + POZ)';

-- Governance
CREATE OR REPLACE ROLE GLOWUP_GOVERNANCE COMMENT = 'Tworzenie i podpinanie polityk';
```

Krok 1.2. Hierarchia ról (SECURITYADMIN)

```
USE ROLE SECURITYADMIN;

GRANT ROLE GLOWUP_READ  TO ROLE GLOWUP_WRITE;
GRANT ROLE GLOWUP_WRITE TO ROLE SYSADMIN;          -- obiekty "rolują się" do SYSADMIN
```

```
GRANT ROLE GLOWUP_READ  TO ROLE GLOWUP_ANALYST;
GRANT ROLE GLOWUP_READ  TO ROLE GLOWUP_MARKETING;
GRANT ROLE GLOWUP_READ  TO ROLE GLOWUP_GOVERNANCE;
GRANT ROLE GLOWUP_WRITE TO ROLE GLOWUP_DPO;
GRANT ROLE GLOWUP_WRITE TO ROLE GLOWUP_SALON_MANAGER;

GRANT ROLE GLOWUP_SALON_MANAGER TO ROLE GLOWUP_MGR_WAW;
GRANT ROLE GLOWUP_SALON_MANAGER TO ROLE GLOWUP_MGR_KRK;
GRANT ROLE GLOWUP_SALON_MANAGER TO ROLE GLOWUP_MGR_GDA;
GRANT ROLE GLOWUP_SALON_MANAGER TO ROLE GLOWUP_MGR_POZ;

GRANT ROLE GLOWUP_MGR_GDA TO ROLE GLOWUP_REGION_NORTH;
GRANT ROLE GLOWUP_MGR_POZ TO ROLE GLOWUP_REGION_NORTH;

GRANT ROLE GLOWUP_GOVERNANCE TO ROLE SYSADMIN;
```

Wskazówka

Celowo **nie** nadajemy ról funkcjonalnych roli SYSADMIN. Polityki z modułów 2 i 3 sprawdzają role funkcją `IS_ROLE_IN_SESSION()`, która uwzględnia dziedziczenie. Gdyby SYSADMIN dziedziczył GLOWUP_DPO, każdy administrator widziałby dane osobowe bez maski. Do SYSADMIN „rolują się" tylko role dostępowe i governance – to wystarcza do zarządzania obiektami.

Krok 1.3. Uprawnienia do obiektów dla ról dostępowych

```
-- SECURITYADMIN (MANAGE GRANTS) może nadawać uprawnienia do obiektów SYSADMIN-a
GRANT USAGE ON WAREHOUSE GLOWUP_WH        TO ROLE GLOWUP_READ;
GRANT USAGE ON DATABASE  GLOWUP           TO ROLE GLOWUP_READ;
GRANT USAGE ON SCHEMA    GLOWUP.CRM       TO ROLE GLOWUP_READ;
GRANT USAGE ON SCHEMA    GLOWUP.SALES     TO ROLE GLOWUP_READ;

GRANT SELECT ON ALL TABLES    IN SCHEMA GLOWUP.CRM   TO ROLE GLOWUP_READ;
GRANT SELECT ON ALL TABLES    IN SCHEMA GLOWUP.SALES TO ROLE GLOWUP_READ;
GRANT SELECT ON FUTURE TABLES IN SCHEMA GLOWUP.CRM   TO ROLE GLOWUP_READ;
GRANT SELECT ON FUTURE TABLES IN SCHEMA GLOWUP.SALES TO ROLE GLOWUP_READ;

GRANT INSERT, UPDATE, DELETE ON ALL TABLES    IN SCHEMA GLOWUP.CRM   TO ROLE GLOWUP_WRITE;
GRANT INSERT, UPDATE, DELETE ON ALL TABLES    IN SCHEMA GLOWUP.SALES TO ROLE GLOWUP_WRITE;
GRANT INSERT, UPDATE, DELETE ON FUTURE TABLES IN SCHEMA GLOWUP.CRM   TO ROLE GLOWUP_WRITE;
GRANT INSERT, UPDATE, DELETE ON FUTURE TABLES IN SCHEMA GLOWUP.SALES TO ROLE GLOWUP_WRITE;

-- Governance zakłada własny schemat na polityki
GRANT CREATE SCHEMA ON DATABASE GLOWUP TO ROLE GLOWUP_GOVERNANCE;

-- Uprawnienia na poziomie konta do podpinania polityk
USE ROLE ACCOUNTADMIN;
GRANT APPLY MASKING POLICY    ON ACCOUNT TO ROLE GLOWUP_GOVERNANCE;
GRANT APPLY ROW ACCESS POLICY ON ACCOUNT TO ROLE GLOWUP_GOVERNANCE;
```

Krok 1.4. Użytkownicy – konta osobowe i konto techniczne (USERADMIN)

```
USE ROLE USERADMIN;

CREATE OR REPLACE USER ANNA_NOWAK TYPE = PERSON
  FIRST_NAME = 'Anna' LAST_NAME = 'Nowak' EMAIL = 'anna.nowak@glowup.example'
  DEFAULT_ROLE = GLOWUP_MGR_WAW DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: kierowniczka salonu Warszawa';

CREATE OR REPLACE USER PIOTR_WISNIEWSKI TYPE = PERSON
  FIRST_NAME = 'Piotr' LAST_NAME = 'Wisniewski' EMAIL = 'piotr.wisniewski@glowup.example'
  DEFAULT_ROLE = GLOWUP_MGR_KRK DEFAULT_WAREHOUSE = GLOWUP_WH
```

```
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: kierownik salonu Krakow';

CREATE OR REPLACE USER MARTA_ZIELINSKA TYPE = PERSON
  FIRST_NAME = 'Marta' LAST_NAME = 'Zielinska' EMAIL = 'marta.zielinska@glowup.example'
  DEFAULT_ROLE = GLOWUP_REGION_NORTH DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: dyrektor regionu Polnoc';

CREATE OR REPLACE USER TOMASZ_LEWANDOWSKI TYPE = PERSON
  FIRST_NAME = 'Tomasz' LAST_NAME = 'Lewandowski' EMAIL = 'tomasz.lewandowski@glowup.example'
  DEFAULT_ROLE = GLOWUP_ANALYST DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: analityk centrali';

CREATE OR REPLACE USER EWA_KAMINSKA TYPE = PERSON
  FIRST_NAME = 'Ewa' LAST_NAME = 'Kaminska' EMAIL = 'ewa.kaminska@glowup.example'
  DEFAULT_ROLE = GLOWUP_MARKETING DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: specjalistka ds. marketingu';

CREATE OR REPLACE USER JAN_WOJCIK TYPE = PERSON
  FIRST_NAME = 'Jan' LAST_NAME = 'Wojcik' EMAIL = 'iod@glowup.example'
  DEFAULT_ROLE = GLOWUP_DPO DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: inspektor ochrony danych';

CREATE OR REPLACE USER KAROL_STAZYSTA TYPE = PERSON
  FIRST_NAME = 'Karol' LAST_NAME = 'Stazysta' EMAIL = 'karol.staz@glowup.example'
  DEFAULT_ROLE = GLOWUP_ANALYST DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: stazysta w centrali';

-- Konto techniczne narzędzia BI: TYPE = SERVICE nie może logować się hasłem
-- (w praktyce: RSA_PUBLIC_KEY = '...' albo OAuth)
CREATE OR REPLACE USER SVC_POWERBI TYPE = SERVICE
  DEFAULT_ROLE = GLOWUP_ANALYST DEFAULT_WAREHOUSE = GLOWUP_WH
  DEFAULT_SECONDARY_ROLES = ()
  COMMENT = 'GlowUp: konto techniczne raportow BI';
```

Krok 1.5. Nadanie ról użytkownikom – także sobie (do testów)

Nie będziemy logować się jako każdy z pracowników. Nadajemy sobie ich role i przełączamy się poleceniem `USE ROLE` – tak samo testuje się polityki w praktyce.

```
USE ROLE SECURITYADMIN;

GRANT ROLE GLOWUP_MGR_WAW      TO USER ANNA_NOWAK;
GRANT ROLE GLOWUP_MGR_KRK      TO USER PIOTR_WISNIEWSKI;
GRANT ROLE GLOWUP_REGION_NORTH TO USER MARTA_ZIELINSKA;
GRANT ROLE GLOWUP_ANALYST      TO USER TOMASZ_LEWANDOWSKI;
GRANT ROLE GLOWUP_MARKETING    TO USER EWA_KAMINSKA;
GRANT ROLE GLOWUP_DPO          TO USER JAN_WOJCIK;
GRANT ROLE GLOWUP_ANALYST      TO USER KAROL_STAZYSTA;
GRANT ROLE GLOWUP_ANALYST      TO USER SVC_POWERBI;

-- Role do testów dla Ciebie
SET me = CURRENT_USER();
GRANT ROLE GLOWUP_ANALYST      TO USER IDENTIFIER($me);
GRANT ROLE GLOWUP_MARKETING    TO USER IDENTIFIER($me);
GRANT ROLE GLOWUP_DPO          TO USER IDENTIFIER($me);
GRANT ROLE GLOWUP_MGR_WAW      TO USER IDENTIFIER($me);
GRANT ROLE GLOWUP_MGR_KRK      TO USER IDENTIFIER($me);
GRANT ROLE GLOWUP_REGION_NORTH TO USER IDENTIFIER($me);
GRANT ROLE GLOWUP_GOVERNANCE   TO USER IDENTIFIER($me);
```

Krok 1.6. Cykl życia konta: koniec stażu

Stażysta Karol kończy praktyki. Konto blokujemy (a nie usuwamy od razu), żeby zachować historię zapytań i audyt, oraz odbieramy role.

```
USE ROLE SECURITYADMIN;
REVOKE ROLE GLOWUP_ANALYST FROM USER KAROL_STAZYSTA;

USE ROLE USERADMIN;
ALTER USER KAROL_STAZYSTA SET DISABLED = TRUE
  COMMENT = 'GlowUp: stazysta - konto zablokowane po zakonczeniu stazu';

-- Inne typowe operacje (do pokazania, nie uruchamiaj):
-- ALTER USER ANNA_NOWAK SET DEFAULT_ROLE = GLOWUP_MGR_WAW;
-- ALTER USER ANNA_NOWAK RESET PASSWORD;        -- link do ustawienia hasła
-- ALTER USER ANNA_NOWAK SET MINS_TO_UNLOCK = 0; -- odblokowanie po błędnych logowaniach
```

Krok 1.7. Uprawnienia FUTURE w działaniu

```
USE ROLE SYSADMIN;
CREATE OR REPLACE TABLE GLOWUP.CRM.LOYALTY_TIERS (
  tier VARCHAR(10), min_visits INT, discount_pct INT);
INSERT INTO GLOWUP.CRM.LOYALTY_TIERS VALUES
  ('BRONZE', 3, 5), ('SILVER', 6, 10), ('GOLD', 10, 15), ('PLATINUM', 20, 20);

USE SECONDARY ROLES NONE;
USE ROLE GLOWUP_ANALYST;
SELECT * FROM GLOWUP.CRM.LOYALTY_TIERS;   -- działa bez dodatkowego GRANT
```

Krok 1.8. Test uprawnień i audyt nadań

```
USE SECONDARY ROLES NONE;
USE ROLE GLOWUP_MARKETING;
SELECT * FROM GLOWUP.CRM.CLIENTS LIMIT 5;     -- działa... i widać wszystkie dane osobowe!
DELETE FROM GLOWUP.CRM.CLIENTS;               -- błąd: Insufficient privileges

USE ROLE SECURITYADMIN;
SHOW GRANTS TO ROLE GLOWUP_READ;              -- co rola może
SHOW GRANTS OF ROLE GLOWUP_SALON_MANAGER;     -- komu rola jest nadana
SHOW GRANTS TO USER MARTA_ZIELINSKA;          -- jakie role ma użytkownik
SHOW FUTURE GRANTS IN SCHEMA GLOWUP.CRM;
```

Wniosek: RBAC kontroluje, **do jakich obiektów** rola ma dostęp, ale nie **które wiersze i kolumny** widzi. Marketing ma dostęp do PESEL-i i danych o zdrowiu – to rozwiążą moduły 2 i 3.

Weryfikacja modułu 1

```
USE SECONDARY ROLES NONE;
USE ROLE SECURITYADMIN;

SHOW USERS;
SET (v_users, v_disabled) = (
  SELECT COUNT(*), COUNT_IF("disabled"::VARCHAR ILIKE 'true')
  FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
  WHERE "comment" ILIKE 'GlowUp:%');

SHOW GRANTS TO ROLE GLOWUP_REGION_NORTH;
SET v_north = (
  SELECT COUNT(*) FROM TABLE(RESULT_SCAN(LAST_QUERY_ID()))
  WHERE "granted_on" = 'ROLE' AND "name" IN ('GLOWUP_MGR_GDA', 'GLOWUP_MGR_POZ'));

SHOW GRANTS TO USER KAROL_STAZYSTA;
SET v_karol = (SELECT COUNT(*) FROM TABLE(RESULT_SCAN(LAST_QUERY_ID())));
```

```
USE ROLE GLOWUP_MARKETING;
SET v_mkt = (SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_ANALYST;
SET v_tiers = (SELECT COUNT(*) FROM GLOWUP.CRM.LOYALTY_TIERS);

SELECT '1.1 8 kont GlowUp, w tym 1 zablokowane' AS test,
       IFF($v_users = 8 AND $v_disabled = 1, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '1.2 REGION_NORTH dziedziczy MGR_GDA i MGR_POZ', IFF($v_north = 2, 'PASS', 'FAIL')
UNION ALL
SELECT '1.3 Zablokowany stazysta nie ma zadnej roli', IFF($v_karol = 0, 'PASS', 'FAIL')
UNION ALL
SELECT '1.4 MARKETING czyta CRM.CLIENTS (1200 wierszy)', IFF($v_mkt = 1200, 'PASS', 'FAIL')
UNION ALL
SELECT '1.5 FUTURE GRANTS: ANALYST czyta nowa tabele (4 wiersze)',
       IFF($v_tiers = 4, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli 1.4 lub 1.5 kończy się błędem „Insufficient privileges”, sprawdź krok 1.3 (USAGE na warehouse, bazie i schemacie). Jeśli `GRANT … TO USER IDENTIFIER($me)` zwraca błąd, nazwa użytkownika zawiera znaki specjalne – wpisz ją ręcznie w cudzysłowie, np. `TO USER "jan.kowalski@firma.pl"`.

4. Moduł 2. Dynamic Data Masking – maskowanie kolumn

Cel modułu

Tworzysz polityki maskowania dla różnych typów danych (tekst, data), stosujesz maskowanie częściowe i warunkowe (zależne od innej kolumny), podpinasz polityki do kolumn i sprawdzasz efekt w kilku rolach. Rozumiesz, że maskowanie działa w czasie zapytania, a dane w tabeli pozostają nienaruszone.

Krok 2.1. Schemat na polityki (rozdzielenie obowiązków)

Polityki tworzy i podpina **GLOWUP_GOVERNANCE**, a nie właściciel tabel. Administrator tabel nie może więc po cichu zdjąć maski, a zespół governance nie musi być właścicielem danych.

```
USE SECONDARY ROLES NONE;
USE ROLE GLOWUP_GOVERNANCE;
USE WAREHOUSE GLOWUP_WH;

CREATE SCHEMA IF NOT EXISTS GLOWUP.SECURITY;
USE SCHEMA GLOWUP.SECURITY;
```

Krok 2.2. Polityki maskowania

```
-- E-mail: maskowanie warunkowe - marketing widzi adres tylko przy zgodzie
CREATE OR REPLACE MASKING POLICY GLOWUP.SECURITY.MP_EMAIL
  AS (val VARCHAR, consent BOOLEAN) RETURNS VARCHAR ->
  CASE
    WHEN IS_ROLE_IN_SESSION('GLOWUP_DPO')
      OR IS_ROLE_IN_SESSION('GLOWUP_SALON_MANAGER')       THEN val
    WHEN IS_ROLE_IN_SESSION('GLOWUP_MARKETING') AND consent THEN val
    WHEN IS_ROLE_IN_SESSION('GLOWUP_ANALYST')
      THEN REGEXP_REPLACE(val, '^[^@]+', '*****')          -- zostaje domena
    ELSE '*** UKRYTE ***'
  END
  COMMENT = 'Email: pelny dla IOD/kierownikow, marketing tylko ze zgoda';

-- Telefon: częściowe maskowanie
CREATE OR REPLACE MASKING POLICY GLOWUP.SECURITY.MP_PHONE
  AS (val VARCHAR) RETURNS VARCHAR ->
  CASE
    WHEN IS_ROLE_IN_SESSION('GLOWUP_DPO')
      OR IS_ROLE_IN_SESSION('GLOWUP_SALON_MANAGER') THEN val
    ELSE '*** *** ' || RIGHT(val, 3)
  END;

-- PESEL: trzy poziomy
CREATE OR REPLACE MASKING POLICY GLOWUP.SECURITY.MP_PESEL
  AS (val VARCHAR) RETURNS VARCHAR ->
  CASE
    WHEN IS_ROLE_IN_SESSION('GLOWUP_DPO')           THEN val
    WHEN IS_ROLE_IN_SESSION('GLOWUP_SALON_MANAGER') THEN REPEAT('*', 7) || RIGHT(val, 4)
    ELSE REPEAT('*', 11)
  END;

-- Data urodzenia: typ zwracany musi być taki sam jak typ kolumny (DATE)
CREATE OR REPLACE MASKING POLICY GLOWUP.SECURITY.MP_BIRTH_DATE
  AS (val DATE) RETURNS DATE ->
  CASE
    WHEN IS_ROLE_IN_SESSION('GLOWUP_DPO')
      OR IS_ROLE_IN_SESSION('GLOWUP_SALON_MANAGER') THEN val
    ELSE DATE_FROM_PARTS(YEAR(val), 1, 1)           -- rok zostaje do analiz wiekowych
  END;

-- Dane o zdrowiu (art. 9 RODO): poza IOD i salonem - brak wartości
CREATE OR REPLACE MASKING POLICY GLOWUP.SECURITY.MP_HEALTH
```

```
  AS (val VARCHAR) RETURNS VARCHAR ->
  CASE
    WHEN IS_ROLE_IN_SESSION('GLOWUP_DPO')
      OR IS_ROLE_IN_SESSION('GLOWUP_SALON_MANAGER') THEN val
    ELSE NULL
  END;
```

Krok 2.3. Podpięcie polityk do kolumn

```
ALTER TABLE GLOWUP.CRM.CLIENTS MODIFY
  COLUMN email         SET MASKING POLICY GLOWUP.SECURITY.MP_EMAIL
                       USING (email, marketing_consent),
  COLUMN phone         SET MASKING POLICY GLOWUP.SECURITY.MP_PHONE,
  COLUMN pesel         SET MASKING POLICY GLOWUP.SECURITY.MP_PESEL,
  COLUMN birth_date    SET MASKING POLICY GLOWUP.SECURITY.MP_BIRTH_DATE,
  COLUMN allergy_notes SET MASKING POLICY GLOWUP.SECURITY.MP_HEALTH;
```

Klauzula `USING (email, marketing_consent)` przekazuje do polityki drugą kolumnę – to **maskowanie warunkowe**. Pierwsza kolumna na liście to zawsze kolumna maskowana.

Krok 2.4. Ta sama tabela, pięć różnych widoków

Uruchom to samo zapytanie kolejno w pięciu rolach i porównaj wyniki z macierzą z rozdziału 1.2:

```
USE SECONDARY ROLES NONE;

USE ROLE GLOWUP_ANALYST;     -- potem: GLOWUP_MARKETING, GLOWUP_MGR_KRK, GLOWUP_DPO, SYSADMIN
SELECT client_id, first_name, email, phone, pesel, birth_date,
       allergy_notes, marketing_consent
FROM GLOWUP.CRM.CLIENTS
WHERE client_id IN ('C00010', 'C00030')
ORDER BY client_id;
```

Rola	email (C00010 / C00030)	pesel	allergy_notes
GLOWUP_ANALYST	*****@… / *****@…	**********	NULL
GLOWUP_MARKETING	*** UKRYTE *** / pełny	**********	NULL
GLOWUP_MGR_KRK	pełny / pełny	*******1234	widoczne
GLOWUP_DPO	pełny / pełny	pełny	widoczne
SYSADMIN	*** UKRYTE *** / *** UKRYTE ***	**********	NULL

Krok 2.5. Skutki uboczne maskowania – warto znać

```
USE ROLE GLOWUP_ANALYST;

-- 1) Filtr działa na wartości zamaskowanej: 0 wierszy, choć klientka istnieje
SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS WHERE email LIKE 'anna.%';

-- 2) Agregaty na kolumnie maskowanej dają mylące wyniki
SELECT COUNT(DISTINCT email) AS unikalnych_adresow FROM GLOWUP.CRM.CLIENTS;   -- ok. 5!

-- 3) Kolumny bez maski można normalnie analizować
SELECT home_salon_id, YEAR(birth_date) AS rocznik, COUNT(*)
FROM GLOWUP.CRM.CLIENTS GROUP BY 1, 2 ORDER BY 1, 2 LIMIT 10;
```

Krok 2.6. Gdzie podpięto polityki?

```
USE ROLE GLOWUP_GOVERNANCE;

SELECT policy_name, policy_kind, ref_entity_name, ref_column_name, ref_arg_column_names
```

```
FROM TABLE(GLOWUP.INFORMATION_SCHEMA.POLICY_REFERENCES(
       REF_ENTITY_NAME => 'GLOWUP.CRM.CLIENTS', REF_ENTITY_DOMAIN => 'table'));

DESCRIBE MASKING POLICY GLOWUP.SECURITY.MP_EMAIL;
```

Weryfikacja modułu 2

```
USE SECONDARY ROLES NONE;

USE ROLE GLOWUP_ANALYST;
SET (a_email, a_pesel, a_allergy, a_bad_dates) = (
  SELECT MAX(IFF(client_id = 'C00030', email, NULL)),
         MAX(IFF(client_id = 'C00030', pesel, NULL)),
         COUNT(allergy_notes),
         COUNT_IF(MONTH(birth_date) <> 1 OR DAY(birth_date) <> 1)
  FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_MARKETING;
SET (m_visible, m_hidden) = (
  SELECT COUNT_IF(email NOT LIKE '%*%'), COUNT_IF(email = '*** UKRYTE ***')
  FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_MGR_KRK;
SET (k_pesel, k_allergy) = (
  SELECT COUNT_IF(RLIKE(pesel, '[*]{7}[0-9]{4}')), COUNT(allergy_notes)
  FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_DPO;
SET (d_pesel, d_allergy) = (
  SELECT COUNT_IF(RLIKE(pesel, '[0-9]{11}')), COUNT(allergy_notes)
  FROM GLOWUP.CRM.CLIENTS);

USE ROLE SYSADMIN;
SET s_hidden = (SELECT COUNT_IF(email = '*** UKRYTE ***') FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_GOVERNANCE;
SET g_refs = (SELECT COUNT(*) FROM TABLE(GLOWUP.INFORMATION_SCHEMA.POLICY_REFERENCES(
                REF_ENTITY_NAME => 'GLOWUP.CRM.CLIENTS', REF_ENTITY_DOMAIN => 'table'))
              WHERE policy_kind = 'MASKING_POLICY');

SELECT '2.1 ANALYST: email z domena, PESEL ukryty, brak alergii, daty = 1 stycznia' AS test,
       IFF($a_email LIKE '*****@%' AND $a_pesel = '**********'
           AND $a_allergy = 0 AND $a_bad_dates = 0, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '2.2 MARKETING: 400 adresow widocznych (zgoda), 800 ukrytych',
       IFF($m_visible = 400 AND $m_hidden = 800, 'PASS', 'FAIL')
UNION ALL
SELECT '2.3 MGR: PESEL czesciowy (1200), alergie widoczne (120)',
       IFF($k_pesel = 1200 AND $k_allergy = 120, 'PASS', 'FAIL')
UNION ALL
SELECT '2.4 DPO: pelne dane (1200 PESEL, 120 alergii)',
       IFF($d_pesel = 1200 AND $d_allergy = 120, 'PASS', 'FAIL')
UNION ALL
SELECT '2.5 SYSADMIN (wlasciciel) widzi maske; 5 kolumn z polityka',
       IFF($s_hidden = 1200 AND $g_refs = 5, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli wszystkie role widzą pełne dane, w sesji są aktywne role drugorzędne – wykonaj `USE SECONDARY ROLES NONE;` i powtórz. Test 2.3 na tym etapie liczy wszystkie 1200 wierszy, bo polityka wierszy pojawi się dopiero w module 3.

5. Moduł 3. Row Access Policies – bezpieczeństwo wierszy

Cel modułu

Tworzysz politykę dostępu do wierszy opartą na tabeli mapującej i hierarchii ról, podpinasz jedną politykę do kilku tabel, zmieniasz uprawnienia bez modyfikowania polityki (zastępstwo urlopowe) i wiesz, że polityka obowiązuje także właściciela tabeli oraz operacje DML.

Krok 3.1. Tabela mapująca: rola → salon

```
USE SECONDARY ROLES NONE;
USE ROLE GLOWUP_GOVERNANCE;
USE SCHEMA GLOWUP.SECURITY;

CREATE OR REPLACE TABLE GLOWUP.SECURITY.SALON_ACCESS (
  role_name VARCHAR(60),
  salon_id  VARCHAR(3),
  valid_note VARCHAR(100)
);

INSERT INTO GLOWUP.SECURITY.SALON_ACCESS VALUES
  ('GLOWUP_MGR_WAW', 'WAW', 'stale'),
  ('GLOWUP_MGR_KRK', 'KRK', 'stale'),
  ('GLOWUP_MGR_GDA', 'GDA', 'stale'),
  ('GLOWUP_MGR_POZ', 'POZ', 'stale');
```

W mapowaniu nie ma roli `GLOWUP_REGION_NORTH` – i nie musi jej być. Dziedziczy ona role MGR_GDA i MGR_POZ, a `IS_ROLE_IN_SESSION()` uwzględnia hierarchię.

Krok 3.2. Polityka dostępu do wierszy

```
CREATE OR REPLACE ROW ACCESS POLICY GLOWUP.SECURITY.RAP_SALON
  AS (p_salon VARCHAR) RETURNS BOOLEAN ->
  -- role centralne widzą wszystkie salony
  IS_ROLE_IN_SESSION('GLOWUP_DPO')
  OR IS_ROLE_IN_SESSION('GLOWUP_ANALYST')
  OR IS_ROLE_IN_SESSION('GLOWUP_MARKETING')
  -- pozostali: tylko salony przypisane w tabeli mapującej
  OR EXISTS (
       SELECT 1
       FROM GLOWUP.SECURITY.SALON_ACCESS m
       WHERE m.salon_id = p_salon
         AND IS_ROLE_IN_SESSION(m.role_name)
     )
  COMMENT = 'Dostep do wierszy wg salonu';

-- Jedna polityka, dwie tabele, różne nazwy kolumn
ALTER TABLE GLOWUP.CRM.CLIENTS  ADD ROW ACCESS POLICY GLOWUP.SECURITY.RAP_SALON
  ON (home_salon_id);
ALTER TABLE GLOWUP.SALES.VISITS ADD ROW ACCESS POLICY GLOWUP.SECURITY.RAP_SALON
  ON (salon_id);
```

Krok 3.3. Kto co widzi?

```
USE SECONDARY ROLES NONE;

USE ROLE GLOWUP_MGR_KRK;         -- potem: GLOWUP_REGION_NORTH, GLOWUP_ANALYST, SYSADMIN
SELECT c.home_salon_id,
       COUNT(DISTINCT c.client_id) AS klientek,
       COUNT(v.visit_id)           AS wizyt,
       SUM(v.amount)               AS obrot
FROM GLOWUP.CRM.CLIENTS c
LEFT JOIN GLOWUP.SALES.VISITS v ON v.client_id = c.client_id
```

```
GROUP BY 1 ORDER BY 1;
```

Oczekiwany wynik: MGR_KRK – tylko KRK (300 klientek, 1500 wizyt). REGION_NORTH – GDA i POZ. ANALYST – cztery salony. **SYSADMIN – pusty wynik**, choć jest właścicielem tabel. Polityka działa przed jakąkolwiek inną logiką zapytania, więc nie da się jej obejść ani JOIN-em, ani widokiem.

Krok 3.4. Zastępstwo urlopowe – zmiana danych, nie kodu

Kierownik z Krakowa idzie na urlop i Anna (MGR_WAW) przejmuje na tydzień także salon KRK. Zespół governance dopisuje jeden wiersz do mapowania – polityka pozostaje bez zmian.

```
USE ROLE GLOWUP_GOVERNANCE;
INSERT INTO GLOWUP.SECURITY.SALON_ACCESS
  VALUES ('GLOWUP_MGR_WAW', 'KRK', 'zastepstwo urlopowe');

USE ROLE GLOWUP_MGR_WAW;
SELECT home_salon_id, COUNT(*) FROM GLOWUP.CRM.CLIENTS GROUP BY 1;   -- WAW 300 + KRK 300

-- Koniec urlopu
USE ROLE GLOWUP_GOVERNANCE;
DELETE FROM GLOWUP.SECURITY.SALON_ACCESS WHERE valid_note = 'zastepstwo urlopowe';

USE ROLE GLOWUP_MGR_WAW;
SELECT home_salon_id, COUNT(*) FROM GLOWUP.CRM.CLIENTS GROUP BY 1;   -- tylko WAW
```

Krok 3.5. Polityka wierszy + maskowanie razem

```
USE ROLE GLOWUP_REGION_NORTH;
SELECT client_id, home_salon_id, email, pesel, allergy_notes
FROM GLOWUP.CRM.CLIENTS
WHERE allergy_notes IS NOT NULL
LIMIT 10;     -- tylko GDA/POZ, PESEL częściowy, alergie widoczne
```

> **Wskazówka**
>
> Tabela mapująca jest odczytywana z uprawnieniami **właściciela polityki** (GLOWUP_GOVERNANCE). Użytkownicy nie potrzebują do niej dostępu – i nie powinni go mieć, bo pokazuje strukturę uprawnień.
>
> Do jednej tabeli można podpiąć tylko jedną politykę wierszy, ale ta może przyjmować kilka kolumn (`ON (salon_id, region)`).
>
> Dla osób z PostgreSQL: RAP to odpowiednik RLS (`CREATE POLICY … USING (…)`), ale w Snowflake właściciel tabeli nie ma wyjątku – nie ma odpowiednika `BYPASSRLS` ani domyślnego zwolnienia właściciela.
>
> Gdy mapowanie jest duże, zapytanie w polityce można zastąpić funkcją z opcją MEMOIZABLE, która buforuje wynik w obrębie zapytania.

Weryfikacja modułu 3

```
USE SECONDARY ROLES NONE;

USE ROLE GLOWUP_MGR_KRK;
SET (k_cli, k_salons) = (SELECT COUNT(*), COUNT(DISTINCT home_salon_id)
                         FROM GLOWUP.CRM.CLIENTS);
SET k_vis = (SELECT COUNT(*) FROM GLOWUP.SALES.VISITS);

USE ROLE GLOWUP_REGION_NORTH;
SET n_cli = (SELECT LISTAGG(DISTINCT home_salon_id, ',') WITHIN GROUP (ORDER BY home_salon_id)
                    || '/' || COUNT(*)
             FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_ANALYST;
SET a_cli = (SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS);
```

```
USE ROLE GLOWUP_MARKETING;
SET m_vis = (SELECT COUNT(*) FROM GLOWUP.SALES.VISITS);

USE ROLE SYSADMIN;
SET s_cli = (SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_MGR_WAW;
SET w_cli = (SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS);

USE ROLE GLOWUP_GOVERNANCE;
SET g_rap = (SELECT COUNT(*) FROM TABLE(GLOWUP.INFORMATION_SCHEMA.POLICY_REFERENCES(
               POLICY_NAME => 'GLOWUP.SECURITY.RAP_SALON')));

SELECT '3.1 MGR_KRK: 300 klientek z 1 salonu, 1500 wizyt' AS test,
       IFF($k_cli = 300 AND $k_salons = 1 AND $k_vis = 1500, 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '3.2 REGION_NORTH (dziedziczenie): GDA,POZ / 600',
       IFF($n_cli = 'GDA,POZ/600', 'PASS', 'FAIL')
UNION ALL
SELECT '3.3 ANALYST 1200 klientek, MARKETING 6000 wizyt',
       IFF($a_cli = 1200 AND $m_vis = 6000, 'PASS', 'FAIL')
UNION ALL
SELECT '3.4 SYSADMIN (wlasciciel) nie widzi zadnego wiersza', IFF($s_cli = 0, 'PASS', 'FAIL')
UNION ALL
SELECT '3.5 RAP na 2 tabelach; zastepstwo usuniete (MGR_WAW = 300)',
       IFF($g_rap = 2 AND $w_cli = 300, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli 3.5 pokazuje 600 dla MGR_WAW, nie usunięto wiersza zastępstwa (krok 3.4). Jeśli tworzenie polityki w kroku 3.2 zgłasza błąd przy `IS_ROLE_IN_SESSION(m.role_name)`, zastąp ten warunek przez `m.role_name = CURRENT_ROLE()`. Wtedy jednak REGION_NORTH potrzebuje własnych wierszy w mapowaniu (GDA i POZ), bo `CURRENT_ROLE()` nie uwzględnia dziedziczenia.

6. Moduł 4. Time Travel – śledztwo i odtwarzanie po incydencie

Cel modułu

Ustawiasz retencję i śledzenie zmian dla tabel z danymi osobowymi, badasz incydent (kto, kiedy, co zmienił) za pomocą historii zapytań i klauzuli CHANGES, odtwarzasz wyłącznie uszkodzone wiersze z przeszłości i sprawdzasz, że Time Travel nie omija polityk maskowania ani polityk wierszy.

Krok 4.1. Retencja i śledzenie zmian

W warsztacie „Pipeline zamówień" korzystaliśmy z domyślnej retencji 1 dnia. Incydent w danych osobowych bywa zauważony po kilku dniach, więc dla tabel CRM ustawiamy 30 dni (edycja Enterprise pozwala na maksymalnie 90). Włączamy też `CHANGE_TRACKING`, które pozwala odpytać zmiany klauzulą CHANGES.

```
USE SECONDARY ROLES NONE;
USE ROLE SYSADMIN;     -- właściciel tabel: może zmieniać parametry, mimo że nie widzi wierszy

ALTER TABLE GLOWUP.CRM.CLIENTS  SET DATA_RETENTION_TIME_IN_DAYS = 30 CHANGE_TRACKING = TRUE;
ALTER TABLE GLOWUP.SALES.VISITS SET DATA_RETENTION_TIME_IN_DAYS = 30 CHANGE_TRACKING = TRUE;

SHOW PARAMETERS LIKE 'DATA_RETENTION_TIME_IN_DAYS' IN TABLE GLOWUP.CRM.CLIENTS;
```

Zmiany CHANGES można odczytać tylko od momentu włączenia śledzenia, dlatego robimy to **przed** incydentem.

Krok 4.2. Incydent: kierownik z Krakowa „porządkuje" dane

Klientka C00009 wycofała zgodę marketingową, a pan Piotr (MGR_KRK) chciał też usunąć jej stare wizyty. Wykonał jednak oba polecenia bez warunku WHERE:

```
USE ROLE GLOWUP_MGR_KRK;

-- miało być: ... WHERE client_id = 'C00009'
UPDATE GLOWUP.CRM.CLIENTS SET marketing_consent = FALSE;
SET q_upd = LAST_QUERY_ID();

-- miało być: ... WHERE client_id = 'C00009'
DELETE FROM GLOWUP.SALES.VISITS;
SET q_del = LAST_QUERY_ID();
```

Zwróć uwagę na liczbę zmienionych wierszy: 300 i 1500, a nie 1200 i 6000. Polityka wierszy ogranicza także UPDATE i DELETE do wierszy widocznych dla roli. Bezpieczeństwo na poziomie wiersza zmniejszyło skalę incydentu czterokrotnie.

Krok 4.3. Śledztwo: kto, kiedy i co

```
USE ROLE GLOWUP_DPO;

-- Kto i kiedy (historia zapytań użytkownika)
SELECT query_id, user_name, role_name, start_time, rows_produced, query_text
FROM TABLE(GLOWUP.INFORMATION_SCHEMA.QUERY_HISTORY(RESULT_LIMIT => 200))
WHERE query_id IN ($q_upd, $q_del);

-- Co dokładnie zmieniono: klientki, które straciły zgodę
SELECT client_id, home_salon_id, marketing_consent AS zgoda_przed,
       METADATA$ACTION, METADATA$ISUPDATE
FROM GLOWUP.CRM.CLIENTS
  CHANGES (INFORMATION => DEFAULT)
  BEFORE (STATEMENT => $q_upd)
WHERE METADATA$ACTION = 'DELETE' AND marketing_consent
ORDER BY client_id;          -- 100 klientek z KRK

-- Jak wyglądał rozkład zgód przed incydentem?
```

```
SELECT home_salon_id, COUNT_IF(marketing_consent) AS zgody
FROM GLOWUP.CRM.CLIENTS BEFORE (STATEMENT => $q_upd)
GROUP BY 1 ORDER BY 1;
```

UPDATE w klauzuli CHANGES pojawia się jako para wierszy: `DELETE` (stara wersja) i `INSERT` (nowa wersja), oba z `METADATA$ISUPDATE = TRUE`. Filtrując wiersze DELETE, widzimy wartości sprzed zmiany. Gdy zmienne `$q_upd` i `$q_del` zostały utracone, ID zapytań znajdziesz w Snowsight: **Monitoring » Query History** (filtr: rola GLOWUP_MGR_KRK).

Krok 4.4. Odtwarzanie punktowe (zamiast CLONE + SWAP)

W warsztacie „Pipeline zamówień" odtwarzaliśmy całą tabelę przez CLONE i SWAP. Tu inni pracownicy od rana dopisują nowe dane, więc przywracamy **tylko uszkodzone wiersze**. Robi to IOD, bo jego rola widzi wszystkie wiersze bez maski i ma uprawnienia do zapisu.

```
USE ROLE GLOWUP_DPO;

-- 1) Przywrócenie zgód (100 wierszy)
UPDATE GLOWUP.CRM.CLIENTS c
SET marketing_consent = h.marketing_consent
FROM GLOWUP.CRM.CLIENTS BEFORE (STATEMENT => $q_upd) h
WHERE c.client_id = h.client_id
  AND c.marketing_consent <> h.marketing_consent;

-- 2) Właściwa operacja: wycofanie zgody klientki C00009
UPDATE GLOWUP.CRM.CLIENTS SET marketing_consent = FALSE WHERE client_id = 'C00009';

-- 3) Przywrócenie usuniętych wizyt (1500 wierszy)
INSERT INTO GLOWUP.SALES.VISITS
SELECT h.*
FROM GLOWUP.SALES.VISITS BEFORE (STATEMENT => $q_del) h
WHERE NOT EXISTS (SELECT 1 FROM GLOWUP.SALES.VISITS v WHERE v.visit_id = h.visit_id);
```

Pułapka: odtwarzanie rolą, która widzi maskę

Maskowanie działa także w `INSERT … SELECT` i `CREATE TABLE … AS SELECT`. Gdyby odtwarzanie wykonała rola, która widzi zamaskowane dane, do tabeli trafiłyby wartości `*** UKRYTE ***` i `**********`, a prawdziwe dane zostałyby nadpisane. Gdyby zaś zrobiła to rola z ograniczonym dostępem do wierszy, odtworzyłaby tylko „swoje" wiersze. Dlatego odtwarzanie zawsze wykonuje rola z pełnym wglądem (tu: GLOWUP_DPO).

Uwaga: krok 2) zmienia zgodę C00009 z TRUE na FALSE, więc w salonie KRK będzie ostatecznie 99 zgód, a w całej bazie 399. Uwzględnia to weryfikacja.

Krok 4.5. Time Travel nie omija zabezpieczeń

```
USE ROLE GLOWUP_ANALYST;
SELECT client_id, email, pesel, allergy_notes
FROM GLOWUP.CRM.CLIENTS BEFORE (STATEMENT => $q_upd)
WHERE client_id = 'C00030';             -- nadal zamaskowane

USE ROLE GLOWUP_MGR_KRK;
SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS BEFORE (STATEMENT => $q_upd);   -- 300, nie 1200
```

Polityki są oceniane w chwili wykonania zapytania, wobec bieżącej roli. Dotyczy to danych bieżących, historycznych (AT/BEFORE), CHANGES oraz klonów – sklonowana tabela zachowuje podpięte polityki.

Wskazówka

RODO i prawo do usunięcia danych: po DELETE dane klientki pozostają odtwarzalne przez okres retencji (tu 30 dni), a potem przez 7 dni Fail-safe. Okres retencji dla tabel z danymi osobowymi warto więc świadomie dobrać i opisać w rejestrze czynności przetwarzania.

Parametr konta `MIN_DATA_RETENTION_TIME_IN_DAYS` (ACCOUNTADMIN) wymusza minimalną retencję, której właściciel tabeli nie może obniżyć – zabezpieczenie przed ustawieniem retencji 0 tuż przed „sprzątaniem".

Pełny audyt dostępu (kto czytał które kolumny) daje widok `SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY` (Enterprise, opóźnienie do 3 godzin).

Weryfikacja modułu 4

```
USE SECONDARY ROLES NONE;

USE ROLE GLOWUP_DPO;
SET (d_cons, d_krk, d_c9) = (
  SELECT COUNT_IF(marketing_consent),
         COUNT_IF(marketing_consent AND home_salon_id = 'KRK'),
         MAX(IFF(client_id = 'C00009', marketing_consent::VARCHAR, NULL))
  FROM GLOWUP.CRM.CLIENTS);
SET (d_vis, d_vis_ids) = (SELECT COUNT(*), COUNT(DISTINCT visit_id) FROM GLOWUP.SALES.VISITS);

USE ROLE GLOWUP_ANALYST;
SET a_tt = (SELECT email FROM GLOWUP.CRM.CLIENTS BEFORE (STATEMENT => $q_upd)
            WHERE client_id = 'C00030');

USE ROLE GLOWUP_MGR_KRK;
SET k_tt = (SELECT COUNT(*) FROM GLOWUP.CRM.CLIENTS BEFORE (STATEMENT => $q_upd));

USE ROLE SYSADMIN;
SHOW PARAMETERS LIKE 'DATA_RETENTION_TIME_IN_DAYS' IN TABLE GLOWUP.CRM.CLIENTS;
SET s_ret = (SELECT "value" FROM TABLE(RESULT_SCAN(LAST_QUERY_ID())));

SELECT '4.1 Zgody odtworzone: 399 lacznie, 99 w KRK, C00009 = false' AS test,
       IFF($d_cons = 399 AND $d_krk = 99 AND $d_c9 = 'false', 'PASS', 'FAIL') AS wynik
UNION ALL
SELECT '4.2 Wizyty odtworzone: 6000 bez duplikatow',
       IFF($d_vis = 6000 AND $d_vis_ids = 6000, 'PASS', 'FAIL')
UNION ALL
SELECT '4.3 Retencja Time Travel CLIENTS = 30 dni', IFF($s_ret = '30', 'PASS', 'FAIL')
UNION ALL
SELECT '4.4 Time Travel nie zdejmuje maski (ANALYST)',
       IFF($a_tt LIKE '*****@%', 'PASS', 'FAIL')
UNION ALL
SELECT '4.5 Time Travel respektuje RAP (MGR_KRK widzi 300)', IFF($k_tt = 300, 'PASS', 'FAIL');
```

Oczekiwany wynik: 5 × PASS. Jeśli 4.1 pokazuje 400/100, nie wykonano kroku 4.4 pkt 2. Jeśli pokazuje 300/0, odtwarzanie wykonano rolą, która nie widzi wszystkich wierszy, albo zmienna `$q_upd` wskazuje inne zapytanie. Jeśli 4.2 pokazuje 7500, INSERT uruchomiono dwa razy bez warunku NOT EXISTS.

7. Podsumowanie i sprzątanie

7.1. Co zbudowaliśmy

Mechanizm	Czego się nauczyliśmy
RBAC	Role systemowe, role dostępowe vs funkcjonalne, hierarchia, FUTURE GRANTS, audyt przez SHOW GRANTS, celowe niepodpinanie ról z dostępem do PII pod SYSADMIN
Użytkownicy	TYPE = PERSON / SERVICE, DEFAULT_ROLE, DEFAULT_SECONDARY_ROLES, blokowanie konta i odbieranie ról, nadawanie ról sobie do testów
Dynamic Data Masking	Maskowanie pełne, częściowe i warunkowe (USING), zgodność typów, skutki dla WHERE i agregatów, POLICY_REFERENCES, rozdzielenie obowiązków
Row Access Policies	Tabela mapująca, IS_ROLE_IN_SESSION z dziedziczeniem, jedna polityka na wielu tabelach, zmiana uprawnień przez dane, brak wyjątku dla właściciela, ograniczenie DML
Time Travel	Retencja 30 dni, CHANGE_TRACKING i CHANGES, śledztwo przez QUERY_HISTORY, odtwarzanie punktowe, polityki obowiązujące także w przeszłości

7.3. Pytania kontrolne

1. Dlaczego SYSADMIN, właściciel tabeli CLIENTS, nie widzi ani jednego wiersza i czy to problem operacyjny?
2. Co by się stało, gdyby GLOWUP_DPO został nadany roli SYSADMIN?
3. Analityk prosi o liczbę unikalnych klientek wg adresu e-mail. Jak mu pomóc, nie zdejmując maski? (Podpowiedź: kolumna client_id, funkcja SHA2 w polityce.)
4. Klientka żąda usunięcia danych. Co robimy z Time Travel i Fail-safe?

7.4. Sprzątanie

```
USE ROLE ACCOUNTADMIN;
DROP DATABASE IF EXISTS GLOWUP;          -- tabele, polityki, mapowanie
DROP WAREHOUSE IF EXISTS GLOWUP_WH;

USE ROLE USERADMIN;
DROP USER IF EXISTS ANNA_NOWAK;          DROP USER IF EXISTS PIOTR_WISNIEWSKI;
DROP USER IF EXISTS MARTA_ZIELINSKA;     DROP USER IF EXISTS TOMASZ_LEWANDOWSKI;
DROP USER IF EXISTS EWA_KAMINSKA;        DROP USER IF EXISTS JAN_WOJCIK;
DROP USER IF EXISTS KAROL_STAZYSTA;      DROP USER IF EXISTS SVC_POWERBI;

DROP ROLE IF EXISTS GLOWUP_REGION_NORTH; DROP ROLE IF EXISTS GLOWUP_MGR_WAW;
DROP ROLE IF EXISTS GLOWUP_MGR_KRK;      DROP ROLE IF EXISTS GLOWUP_MGR_GDA;
DROP ROLE IF EXISTS GLOWUP_MGR_POZ;      DROP ROLE IF EXISTS GLOWUP_SALON_MANAGER;
DROP ROLE IF EXISTS GLOWUP_ANALYST;      DROP ROLE IF EXISTS GLOWUP_MARKETING;
DROP ROLE IF EXISTS GLOWUP_DPO;          DROP ROLE IF EXISTS GLOWUP_GOVERNANCE;
DROP ROLE IF EXISTS GLOWUP_WRITE;        DROP ROLE IF EXISTS GLOWUP_READ;
```